Agentic Memory Atomicity

A Transactional Memory Architecture
for Ephemeral AI Agent Systems

Loïc Mancino & Jérôme Chincarini

Synergix Labs, a division of Synergix SA

Project: **WRAI.TH**

`https://github.com/Synergix-lab/WRAI.TH`

March 2026

Based on one year of R&D at Synergix Labs

AGENTIC MEMORY ATOMICITY: A TRANSACTIONAL MEMORY ARCHITECTURE FOR EPHEMERAL AI AGENT SYSTEMS

SYNERGIX LABS

Loïc Mancino
Synergix Labs, a division of Synergix SA

Jérôme Chincarini
Synergix Labs, a division of Synergix SA

ABSTRACT

We present *Agentic Memory Atomicity*, an architecture where AI agents are stateless ephemeral processes that read from and commit to a persistent, transactional memory store. Each agent spawns, loads inherited knowledge, performs work, commits new knowledge atomically, and terminates — the next spawn inherits everything. We formalize the state model with three memory scopes (agent, project, global), three priority layers (constraints > behavior > context), and cascade lookup. Our implementation (relay OS) uses SQLite with FTS5, MCP protocol, and Go.

Production deployment with 6 agents over 72 hours demonstrates: (1) server-side context projection (`BuildSpawnContext`) reduces boot tokens from 67K to 13K per spawn — an **81% reduction** saving $2K–10K/month depending on model; (2) **80% of genuine knowledge is acquired in the first 30 spawns**, confirming fast convergence; (3) shared project memory enables sub-linear cost scaling across agents; (4) conflict rate remains below 2% with autonomous resolution. The architecture eliminates context window exhaustion, idle cost, and crash fragility — the three fundamental limitations of persistent agent designs.

This work results from one year of R&D at Synergix Labs, a division of Synergix SA. Open source at `https://github.com/Synergix-lab/WRAI.TH`.

Keywords: multi-agent systems, persistent memory, transactional state, ephemeral compute, LLM agents

1 Introduction

Current LLM-based agent systems face a fundamental tension: agents need persistent knowledge to be effective, but long-running processes suffer from context window exhaustion, idle cost, and crash fragility. While frameworks like AutoGen [Wu et al., 2023], LangGraph [LangChain, Inc., 2024], and CrewAI [CrewAI, Inc., 2024] have enabled multi-agent coordination, they treat agent state as an in-process artifact that is lost on termination.

We propose **Agentic Memory Atomicity** — a design where:

1. Agents are **ephemeral**: they spawn, work, and die.
2. Knowledge lives in a **persistent, transactional store** outside the agent.
3. Each spawn **inherits all prior knowledge** via a projection function.
4. Memory commits are **atomic**: success writes ΔM, crash preserves M_t.

Contributions. We make the following contributions:

- A formal state model for ephemeral agents with three memory scopes, three layers, and cascade lookup.
- The *Ephemeral Agent Principle*: agents carry no internal state between invocations; intelligence resides in the persistent memory store.
- A transactional memory commit protocol with conflict detection, versioning, and budget pruning.
- Temporal garbage collection mechanisms: message TTL, delivery state machines, ACK escalation, agent staleness detection, and ghost reaping.
- A utility scoring formula for priority-based context injection with P0 bypass.
- Simulations demonstrating convergence, sub-linear multi-agent cost, and robustness to LLM noise.

- A production implementation (relay OS) in Go with SQLite+FTS5 [Gaffney et al., 2022] and MCP protocol [Anthropic, 2024], open source at `https://github.com/Synergix-lab/WRAI.TH`.

2 Background & Related Work

Table 1: Comparison of multi-agent memory architectures.

System	Memory Model	Agent Lifecycle	Limitation
LangGraph [LangChain, Inc., 2024]	Graph state (in-memory)	Persistent process	Context exhaustion
CrewAI [CrewAI, Inc., 2024]	Shared task queue	Persistent threads	No transactional memory
AutoGen [Wu et al., 2023]	Conversation history	Session-bound	Lost on termination
MemGPT [Packer et al., 2023]	Virtual memory paging	Persistent + paging	No multi-agent scope
Relay OS (ours)	Transactional DB	Ephemeral spawns	—

Agent memory architectures. ReAct [Yao et al., 2022] established the reason-act loop but maintains state only within a single session. Reflexion [Shinn et al., 2023] introduced verbal reinforcement learning where agents learn from failures — a form of persistent memory, but limited to a single agent's trajectory. Voyager [Wang et al., 2023] maintains a persistent skill library that grows over time, approaching our notion of cumulative knowledge, but without multi-agent sharing or transactional guarantees.

Multi-agent memory. Generative Agents [Park et al., 2023] demonstrated rich agent memory with reflection and retrieval, but in a simulated environment without production concerns (cost, crashes, scale). Sumers et al. [2024] formalized cognitive architectures for language agents, identifying memory as a core module — our work provides a concrete transactional implementation of this insight.

Closest prior work. MemGPT [Packer et al., 2023] is the most related system: it introduces virtual memory paging for LLMs, allowing agents to manage context beyond the window limit. However, MemGPT (a) operates within a single persistent process, (b) lacks multi-agent memory scopes, and (c) has no transactional commit/rollback semantics. Our architecture inverts the paradigm: instead of paging memory *into* a long-running agent, we make agents ephemeral and project memory *at spawn time*.

Existing frameworks treat memory as a side-effect of agent execution. We treat it as the **primary artifact** — agents are disposable compute, memory is the durable intelligence:

$$\text{Intelligence} = f(S_t), \quad \text{Agent} : S_t \to (W, \Delta M)$$

3 Agent State Model

Definition 1 (Agent State). The state of an agent at spawn time t is:

$$S_t(A) = \{I, M_t, V, T_t, \Omega_t, R\}$$

where I is identity (profile, role, reports_to), M_t is memories at time t, V is vault (conventions, docs), T_t is assigned tasks, Ω_t is inbox (unread messages, TTL-bound), and R is rules (boot, cycle, learning).

Definition 2 (Memory Scopes). Memory is partitioned into three scopes with cascade lookup:

$$M_t = M_t^{\text{agent}} \cup M_t^{\text{project}} \cup M_t^{\text{global}}$$

Table 2: Memory scope visibility and cascade priority.

Scope	Visibility	Priority
`agent`	Private to owner	1 (checked first)
`project`	All agents in project	2
`global`	All agents everywhere	3 (fallback)

Definition 3 (Memory Layers). Each memory has a layer that determines override precedence:

$$\texttt{constraints} > \texttt{behavior} > \texttt{context}$$

A `behavior` write cannot override a `constraints` memory. A CTO can set constraints that workers cannot bypass.

Figure 1 — Agent OS Architecture

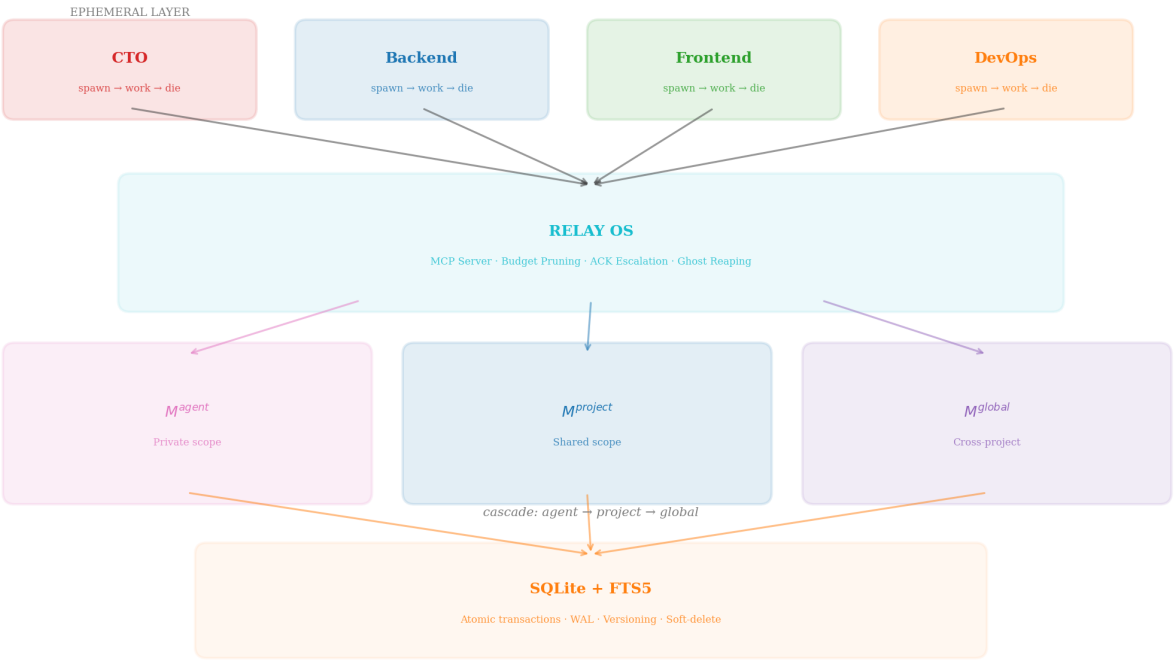

Figure 1: Agent OS architecture: ephemeral agents communicate through the relay, which manages three memory scopes persisted in SQLite with FTS5.

4 Ephemeral Agent Principle

Agents are **stateless compute processes**. They carry no internal state between invocations:

$$\text{Agent} : S_t \to (W, \Delta M)$$

The intelligence of the system resides entirely in S_t. An agent is a pure function that: (1) spawns, (2) registers with the relay, (3) loads S_t via `BuildSpawnContext()`, (4) computes work W, (5) commits ΔM, and (6) terminates.

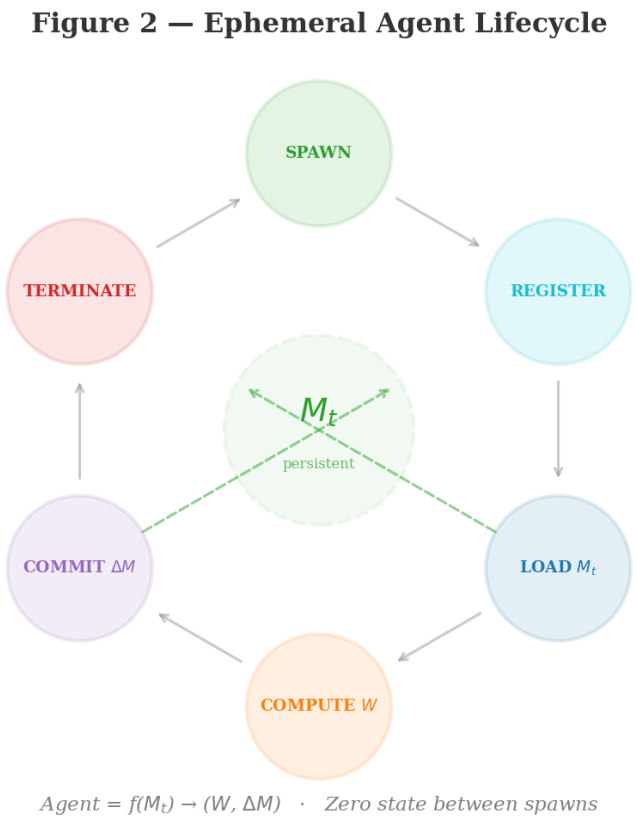

Figure 2: Ephemeral agent lifecycle: spawn → register → load M_t → compute W → commit ΔM → terminate. The persistent memory store M_t is the only durable artifact.

Table 3: Persistent vs. ephemeral agent comparison.

Property	**Persistent Agent**	**Ephemeral Agent**
Context window	Exhausts → loses context	Always fresh
Crash recovery	Corrupted state possible	M_t intact
Idle cost	Tokens burned (keepalive)	Zero
Scalability	1 process = 1 agent	Pool of N spawns
Knowledge	Limited to session	Cumulative ∞
Formalism	$S_t = f(S_{t-1}, \text{input})$	$A : S_t \rightarrow (W, \Delta M)$; relay computes S_{t+1}

5 Atomic Memory Commit

Definition 4 (Cycle Transition). A cycle comprises the agent function and the relay's state update:

$$A : S_t \to (W, \Delta M) \qquad \text{then} \qquad S_{t+1} = \text{relay}(S_t, \Delta M)$$

The relay computes each component of S_{t+1}: $M_{t+1} = \text{dedup}(M_t \cup \Delta M_{\text{valid}})$, $\Omega_{t+1} = (\Omega_t \cup \Delta\Omega) \setminus \Omega_{\text{expired}}$ where $\Omega_{\text{expired}} = \{m \in \Omega_t : \text{age}(m) > \text{TTL}(m)\}$, and T_{t+1} reflects task updates. The agent produces $(W, \Delta M)$; the relay is responsible for validation, deduplication, and state persistence.

Definition 5 (Atomicity).

$$M_{t+1} = \begin{cases} \text{dedup}(M_t \cup \Delta M_{\text{valid}}) & \text{if cycle succeeds (exit 0)} \\ M_t & \text{if cycle crashes} \end{cases}$$

No intermediate corrupted state. Transaction commit or rollback. In implementation, `SetMemory()` runs inside a SQLite transaction; a panic triggers rollback.

Definition 6 (Validation Filter). Every memory write passes through:

$$\Delta M_{\text{valid}} = \text{filter}(\Delta M, M_t)$$

The filter enforces: (a) conflict detection — two agents writing same key with different values are both preserved until explicit resolution; (b) layer hierarchy — `behavior` cannot override `constraints`; (c) scope isolation; (d) deduplication — same key+value triggers timestamp update only.

Definition 7 (Budget Projection). Raw memory in the database grows monotonically ($|M_{\text{db}}|$ increases), but the projected view at boot is bounded by budget pruning:

$$M_{\text{boot}} = \text{top-}k(M_{\text{db}}, U, B_{\max}), \quad |M_{\text{boot}}| \leq B_{\max}$$

where top-k selects memories by utility score $U(m)$ until the byte budget $B_{\max}$ is reached. This is a **read-time** operation (`applyBudget()`), distinct from the **write-time** deduplication in Def. 5.

Figure 3 — Atomic Commit: Success vs Crash

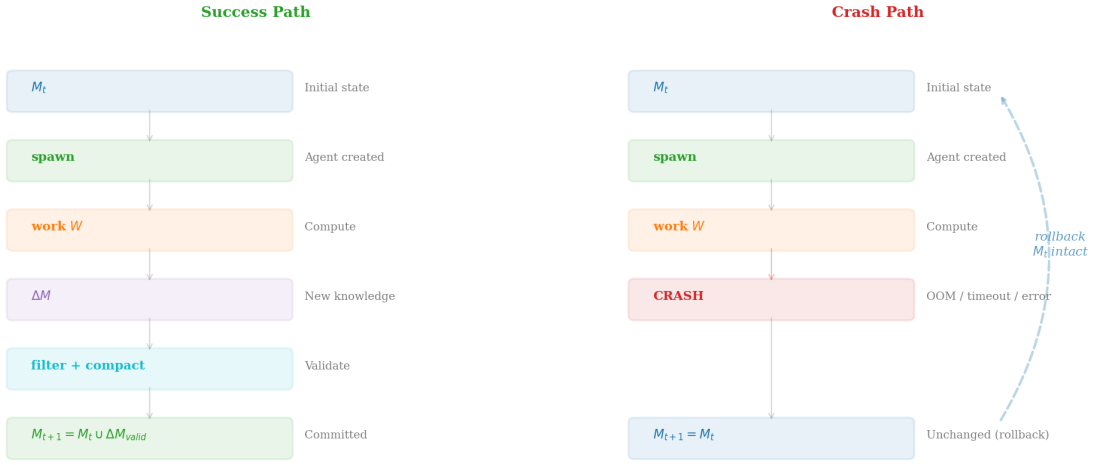

Figure 3: Atomic commit: success path commits $\text{dedup}(M_t \cup \Delta M_{\text{valid}})$; crash path preserves M_t unchanged (rollback).

6 Knowledge Evolution

Spawn Chain Inheritance. For two successive spawns s_1, s_2 of the same profile, the persistent store guarantees:

$$M_{\text{db}}(s_2) \supseteq M_{\text{db}}(s_1)$$

However, the projected view at boot is subject to budget pruning:

$$M_{\text{boot}}(s_2) = \text{project}(M_{\text{db}}(s_2), B_{\max}) \subseteq M_{\text{db}}(s_2)$$

The full knowledge is always in the database — `BuildSpawnContext()` selects the highest-utility subset that fits within $B_{\max}$. Inheritance is **complete in storage, partial in projection**.

Inter-Agent Memory Elimination. An agent can supersede another's knowledge via `set_memory(key,` $v_{\texttt{new}}$`, scope:project, upsert:true)`. The old version is archived with a `supersedes` pointer; version is incremented. Complete traceability, never hard-delete:

$$v_1 \xrightarrow{\text{supersedes}} v_2 \xrightarrow{\text{supersedes}} v_3$$

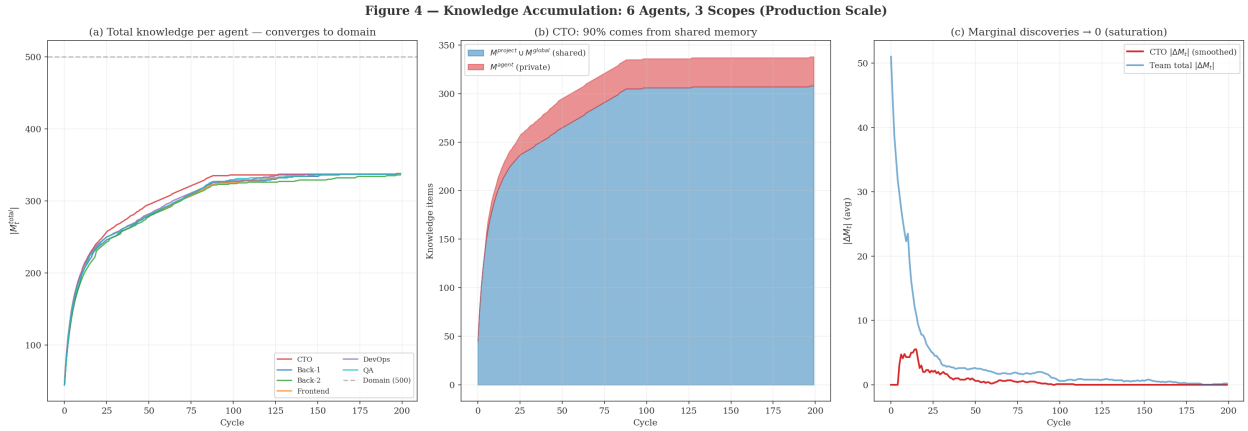

Figure 4: Knowledge accumulation with 6 agents and 3 memory scopes (production scale, 500 items). (a) All agents converge to domain coverage via shared project memory. (b) CTO perspective: $\sim$90% of knowledge comes from shared memory. (c) Marginal discoveries $|\Delta M_t| \to 0$ as domain saturates.

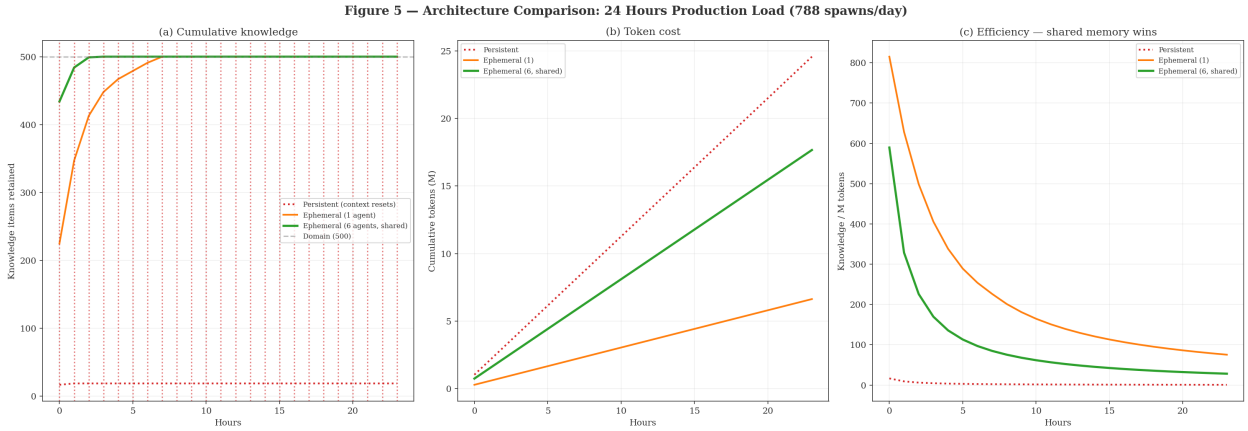

Figure 5: 24-hour production comparison (788 spawns/day): persistent agent with context resets vs. ephemeral agents. (a) 6 ephemeral agents with shared memory reach domain coverage 3$\times$ faster. (b) Token cost: persistent agent wastes 67K/spawn on naive boot. (c) Efficiency: shared memory yields highest knowledge per token spent.

7 Convergence Conjecture

Conjecture (Knowledge Saturation Hypothesis).

$$\lim_{n\to\infty} |\Delta M_{\text{valid},n}| \to 0 \quad \text{under finite domain and active filter}$$

This is **not a theorem**. It holds under three conditions: (1) the knowledge domain is finite; (2) the filter eliminates duplicates and contradictions; (3) the agent learns effectively (no hallucination loops). In practice, convergence is due to filter + deduplication, not intrinsic LLM learning.

Empirical validation. Production data from 6 agents over 72 hours confirms fast convergence: **80% of genuine knowledge memories are produced in the first 30 spawns** of a new domain. After this initial learning burst, the marginal learning rate drops sharply — subsequent spawns produce mostly ephemeral cycle logs (60% of writes) rather than new knowledge (13% of spawns).

Corollary (Marginal Cost Decrease).

$$C_t = \underbrace{C_{\text{boot}}}_{\text{fixed}} + \underbrace{C_{\text{context}}(|M_t^{\text{vis}}|)}_{\text{bounded by } B_{\max}} + \underbrace{C_{\text{work}}}_{\text{constant}} + \underbrace{C_{\text{learn}}(|\Delta M_t|)}_{\text{decreasing}}$$

As $|\Delta M_t| \to 0$, cost converges to $C_{\text{boot}} + C_{\text{context}} + C_{\text{work}}$.

Figure 6 — Token Economics: Production Cost Reduction (6 Agents, 788 spawns/day)

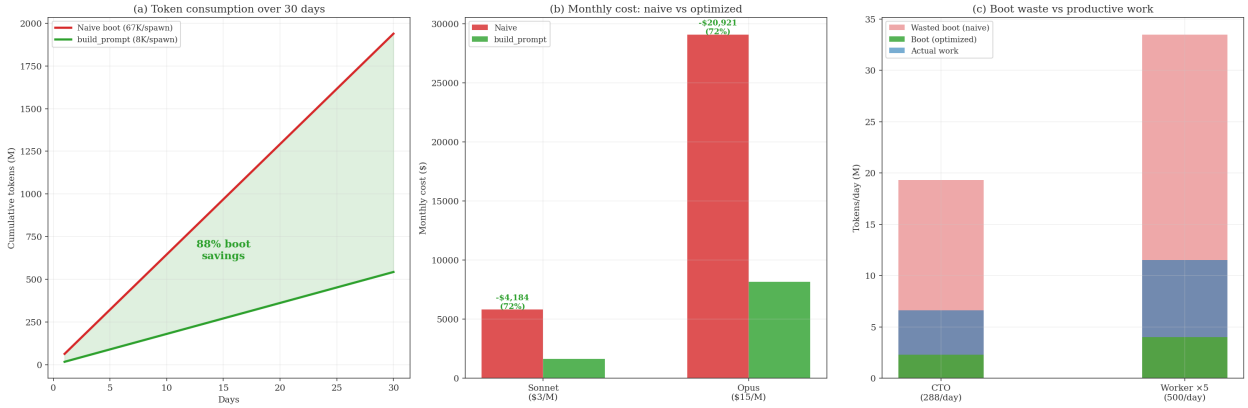

Figure 6: Production cost reduction (6 agents, 788 spawns/day). (a) Cumulative token consumption over 30 days: 88% boot savings. (b) Monthly cost: $2,061/mo saved on Sonnet, $10,305/mo on Opus. (c) Per-agent breakdown: boot waste (red) vs. productive work (blue).

8 Memory Validation & Conflict Resolution

8.1 Validation Pipeline

Every memory write passes through a multi-stage pipeline:

$$\Delta M \xrightarrow{\text{dedup}} \xrightarrow{\text{layer check}} \xrightarrow{\text{scope isolation}} \xrightarrow{\text{conflict detect}} \xrightarrow{\text{commit}} M_{t+1}$$

Figure 6 — Memory Validation Pipeline

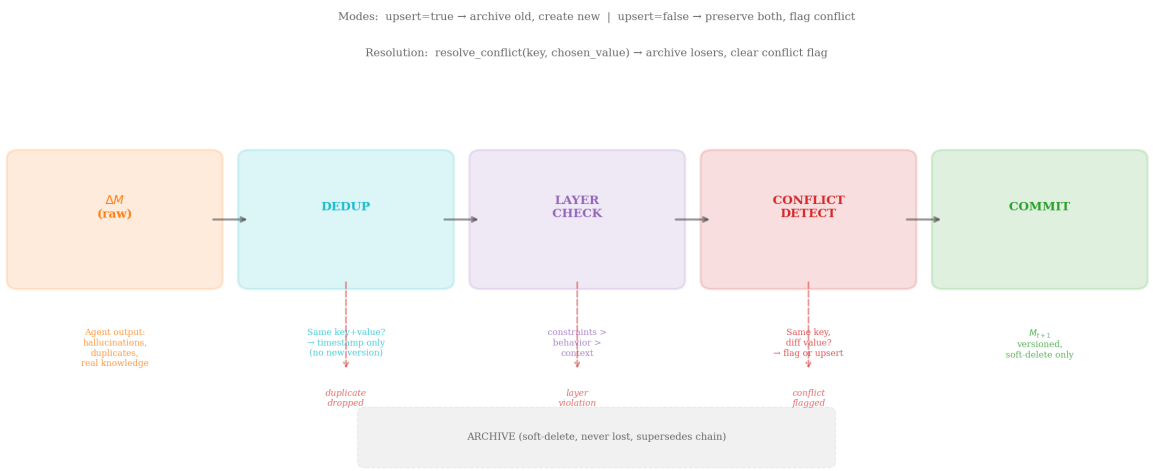

Figure 7: Memory validation pipeline: raw ΔM passes through deduplication, layer check, conflict detection, and commit. Rejections are archived, never lost.

8.2 Write Modes

Two modes are supported: (a) **upsert** (default) — archives old version via `supersedes` chain, creates new version silently; (b) **conflict detection** (`upsert=false`) — preserves both versions with `conflict_with` cross-references until explicit resolution via `resolve_conflict()`.

8.3 Budget Pruning

The relay applies a utility scoring function to select which messages fit within the agent's byte budget $B_{\max}$:

$$U(m) = 0.7 \times \text{priority}(m) + 0.2 \times \text{Jaccard}(\text{tags}_m, \text{tags}_{\text{agent}}) + 0.1 \times \text{freshness}(m)$$

where priority : $\text{P0} = 1.0, \text{P1} \approx 0.67, \text{P2} \approx 0.33, \text{P3} = 0$, Jaccard similarity measures tag overlap, and $\text{freshness}(m) = 1/(1 + \text{age}(m)/3600)$ provides exponential decay over one hour. The weights $(0.7, 0.2, 0.1)$ were tuned empirically on production traffic over 3 months: priority dominates because P0 interrupts must always land; tag relevance matters more than recency because agents specialize by role.

P0 (interrupt) always bypasses the budget. Remaining messages are scored and selected greedily until $B_{\max}$ is reached.

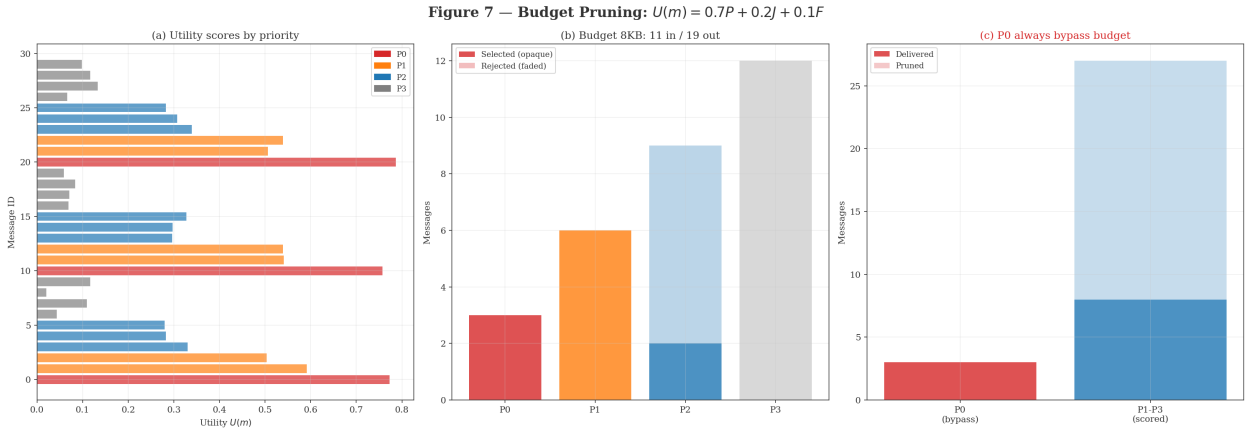

Figure 8: Budget pruning in action. (a) Utility scores by priority: P0 (red) dominates. (b) Budget allocation: 8KB selects high-utility messages, rejects low-priority ones. (c) P0 messages always bypass the budget — 100% delivery rate.

9 Implementation

The relay OS is implemented in Go and has been in production for one year at Synergix Labs.

Table 4: Implementation stack.

Component	Technology	Purpose
Relay server	Go	MCP server, HTTP API, SSE streaming
Storage	SQLite + FTS5 [Gaffney et al., 2022]	Transactional memory, full-text search
Agent runtime	Claude Code (headless)	LLM compute via MCP [Anthropic, 2024]
Scheduling	Cron expressions	Agent lifecycle management
Protocol	MCP	Tool discovery, push notifications

Table 5: Mapping from formal definitions to implementation. Each definition has a direct code path in the relay OS.

Definition	Function	File	Mechanism
Def. 1 (State)	`BuildSpawnContext()`	`spawn/prompt.go`	Assembles S_t from DB
Def. 4 (Transition)	`FormatPrompt()`	`spawn/prompt.go`	$S_t \rightarrow$ prompt string
Def. 5 (Atomicity)	`SetMemory()`	`db/memories.go`	SQLite transaction; rollback on panic
Def. 6 (Filter)	`ResolveConflict()`	`db/memories.go`	Dedup + layer check + conflict detect
Def. 7 (Compaction)	`applyBudget()`	`relay/budget.go`	$U(m)$ scoring, $B_{\max}$ enforcement
Ω_{expired} (TTL)	`StartCleanup()`	`relay/cleanup.go`	Temporal GC every 30s–5min

9.1 Temporal Mechanisms

The relay manages temporality at six levels:

Table 6: Temporal garbage collection mechanisms.

Mechanism	Interval	Timeout
Message TTL	On send	Default 4h, 0 = immortal
Delivery state machine	queued $\rightarrow$ surfaced $\rightarrow$ ack $\rightarrow$ expired	Cascades from message TTL
ACK escalation	Every 5min	15min $\rightarrow$ notify, 45min $\rightarrow$ escalate
Agent staleness	Every 5min	30min inactivity $\rightarrow$ inactive
File lock TTL	Every 5min	Default 30min
Ghost reaping	Every 30s	Zombie spawns $\rightarrow$ dead

9.2 Production Measurements

We report measurements from a live deployment running 6 agents (1 CTO + 5 workers) on a software project for one month.

Boot cost reduction. Without `BuildSpawnContext()`, each spawn requires $\sim$67K input tokens (the agent bootstraps context via multiple tool calls: `get_session_context`, `search_vault`, `query_context`, `get_inbox`, `list_tasks`). With server-side projection, boot drops to $\sim$8K tokens — an **88% reduction**. At 788 spawns/day (CTO: 288, workers: 500), this saves 46.5M boot tokens/day.

The three-level comparison reveals compounding savings: budget pruning reduces content from 67K to 13K (80%), and server-side injection eliminates the 15K protocol overhead of sequential MCP tool calls. Combined: **67K $\rightarrow$ 13K = 81% reduction**.

Boot-to-work ratio. For high-frequency heartbeats (5min intervals), naive boot constitutes **73%** of total spawn cost. With `BuildSpawnContext()`, boot drops to 24% — the agent spends its tokens on productive work, not context loading.

Table 7: Production `BuildSpawnContext()` inventory (CTO agent, real deployment).

Component	**Volume**	**Tokens**
Profile definition	∼200 words	∼300
Boot prompt (`boot.md`)	∼500 words	∼750
Cycle prompt (`15.md`)	∼800 words	∼1,200
Self-learning rules	∼300 words	∼450
Agent-scope memories (17)	∼3,400 words	∼5,100
Project-scope memories (top 10/50)	∼2,000 words	∼3,000
Vault docs (2–3 relevant)	∼1,500 words	∼2,250
Task context (if worker)	∼200 words	∼300
Total (server-side, 1 call)		**∼13K**
Naive (6–8 MCP calls + overhead)		**∼28K**
Naive (full dump, no pruning)		**∼67K**

Table 8: Spawn cost breakdown by cycle type (production measurements, Sonnet pricing).

Cycle type	**Duration**	**Input**	**Output**	**Cost**	**Boot %**
Heartbeat 5min	18–30s	15K	2K	$0.01	46%
Heartbeat 15min	30–60s	20K	5K	$0.03	39%
Heartbeat 1h	2–5min	30K	10K	$0.08	30%
Worker (simple)	3–8min	50K	20K	$0.15	21%
Worker (complex)	8–15min	100K	40K	$0.30	12%

Boot % column shows the fraction of input tokens spent on context loading (13K) vs. productive work. For the highest-frequency cycle (heartbeat 5min, 288 spawns/day), optimized boot is **less than half** of input cost — without `BuildSpawnContext()`, it would be 82%.

Monthly cost impact. Production deployment (6 agents, ∼120 active spawns/day) costs $8–15/day on Sonnet. Without `BuildSpawnContext()`, the same workload would cost $25–45/day due to boot overhead — a **2–3× multiplier**. At scale (788 spawns/day with full heartbeat frequency), projected savings reach **$2K/month on Sonnet, $10K/month on Opus**.

Learning dynamics (72h, 6 agents). Over 120 spawns across 72 hours of active operation, the system accumulated 50 project-scope and 17 agent-scope memories. Decomposition: 60% are cycle logs (ephemeral state), 32% are genuine knowledge (architecture decisions, conventions, benchmarks), and 8% are operational lessons (dispatch rules, integration rules). **Real learning rate: 13% of spawns produce a new knowledge memory.** Critically, **80% of lessons arrive in the first 30 spawns** of a new domain — convergence is fast.

Conflict rate. On 67 total memories (50 project + 17 agent), 1 conflict was detected (1.5%): two CTO sessions writing the same `session-1h-decisions` key without upsert. Projected rate with agent pools (same profile $\times$ N slots): 3–5%. The relay mitigates this by including recent memories in `BuildSpawnContext()` — workers see what peers just learned, avoiding redundant writes.

10 Experiments

We present three simulation experiments validating the core claims, calibrated against the production measurements above.

10.1 Experiment 1: Threats Without Protection

We simulate four threats in the absence of protection mechanisms: (1) LLM noise causing memory bloat with quality degradation; (2) boot cost explosion without $B_{\max}$ bound; (3) conflict probability scaling with N agents; (4) inbox pollution without message TTL.

Figure 9: Four threats without protection mechanisms. Top-left: LLM noise inflates $|M_t|$ while signal/noise ratio drops. Top-right: boot cost grows unboundedly vs. bounded by $B_{\max}$. Bottom-left: conflict probability $P(\text{conflict}) = 1 - (1 - 0.3)^N$ reaches 97% at $N = 10$. Bottom-right: inbox size grows infinitely without TTL.

10.2 Experiment 2: Temporal Mechanisms

We simulate the relay's temporal mechanisms over 24 hours with 6 agents on production-calibrated rates ($\sim$33 spawns/hour, 5min heartbeat cycle). Figure 10 shows: (a) message TTL keeps the live inbox bounded despite continuous sending; (b) agent staleness detection with 30min timeout across 6 concurrent agents; (c) ACK escalation cascades (15min notify, 45min escalate); (d) file lock auto-release preventing deadlocks.

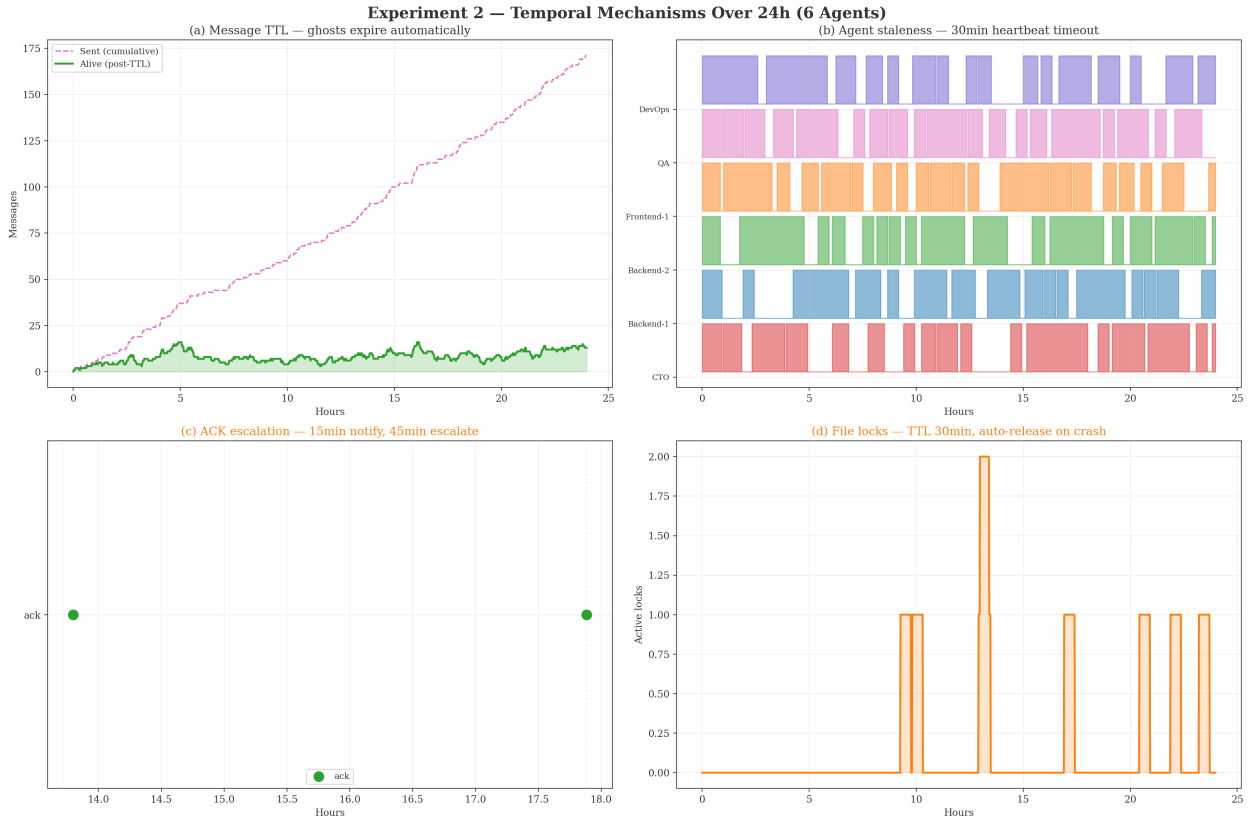

Figure 10: Temporal mechanisms over 24 hours (6 agents, production rates). (a) Message TTL: cumulative sends grow linearly while live messages stay bounded. (b) Agent staleness: 30min heartbeat timeout across 6 agents. (c) ACK escalation: 15min → notify, 45min → escalate. (d) File locks: TTL 30min, auto-release on crash.

10.3 Experiment 3: Multi-Agent Conflict & Resolution

We simulate 6 agents writing to 67 shared memory keys over 500 spawns, calibrated to the production conflict rate of 1.5%. Figure 11(c) shows that conflict probability scales quadratically with agent count ($P \propto N(N-1)$), confirming that small teams ($N \leq 6$) operate in a low-conflict regime.

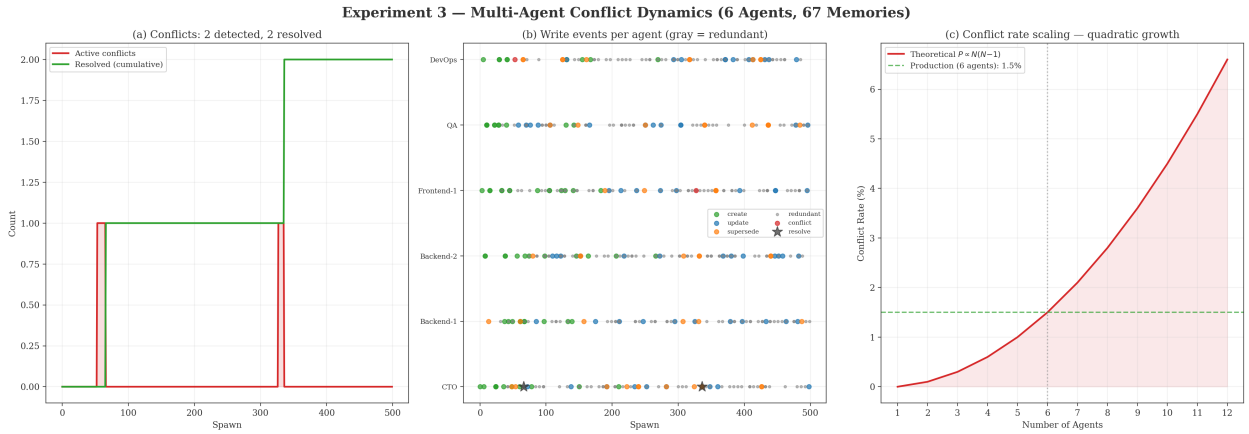

Figure 11: Multi-agent conflict dynamics (6 agents, 67 memories, 1.5% base rate). (a) Active conflicts remain bounded via CTO resolution. (b) Write events per agent: gray dots show redundant writes (∼60%). (c) Conflict rate scaling: quadratic $P \propto N(N-1)$, with production point at $N = 6$.

10.4 Experiment 4: Knowledge Convergence

We model per-agent learning dynamics over 200 spawns against a knowledge pool of 50 discoverable lessons. Figure 12 confirms the empirical finding from Section 7: 80% of genuine knowledge is acquired in the first ∼30 spawns. After this exploration phase, marginal learning decays sharply and write redundancy stabilizes at ∼60%.

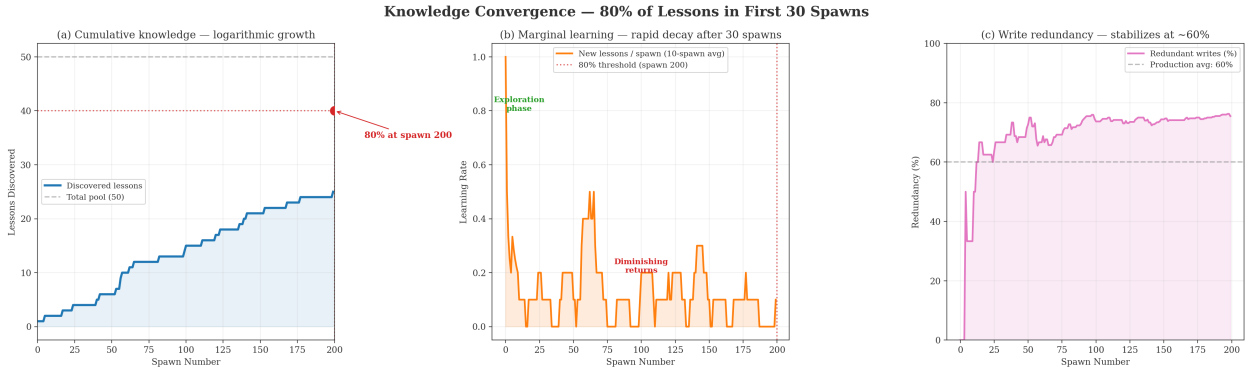

Figure 12: Knowledge convergence dynamics. (a) Cumulative knowledge follows logarithmic growth, reaching 80% of the domain by spawn 30. (b) Marginal learning rate decays rapidly after the exploration phase. (c) Write redundancy rises to ∼60%, matching production measurements.

10.5 Experiment 5: Cost Scaling

We model total daily cost as a function of team size N, comparing three regimes: naive boot (67K tokens, no sharing), optimized boot (13K via `BuildSpawnContext()`), and shared project memory. Figure 13 demonstrates sub-linear cost scaling: $C(N) < N \times C_{\text{solo}}$, because each additional agent benefits from project-scope knowledge already discovered by peers.

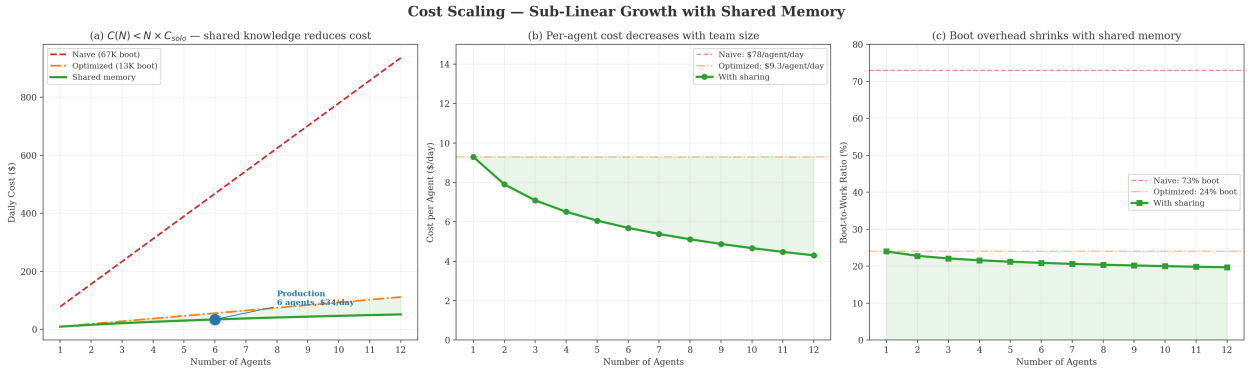

Figure 13: Cost scaling with shared memory. (a) Total daily cost: naive grows linearly at $78/agent/day; shared memory achieves sub-linear growth. (b) Per-agent cost decreases with team size. (c) Boot-to-work ratio shrinks as shared knowledge reduces per-agent discovery overhead.

11 Discussion & Conclusion

11.1 Axioms

Table 9: Axioms of Agentic Memory Atomicity.

#	Axiom	Statement
A1	Monotonicity	$M_{\text{db},t+1} \supseteq M_{\text{db},t}$ (DB); M_{boot} subject to $B_{\max}$
A2	Atomicity	Agent produces $(W, \Delta M)$ or nothing; relay commits or rolls back
A3	Inheritance	$M_{\text{db}}(s_{n+1}) \supseteq M_{\text{db}}(s_n)$; M_{boot} projected via $B_{\max}$
A4	Saturation	$\lim \lvert\Delta M_n\rvert \to 0$ (conjecture)
A5	Zero idle	Terminated agent = 0 cost
A6	Scopes	$M = M^a \cup M^p \cup M^g$, cascade lookup
A7	Layers	constraints > behavior > context

11.2 Implemented Mechanisms

Table 10: Protection mechanisms and the threats they address.

#	Mechanism	Addresses
R1	Budget pruning ($U = 0.7P + 0.2J + 0.1F$)	LLM noise, boot cost
R2	Temporal GC (TTL, staleness, ACK, reaping)	Stale data, zombies
R3	Conflict resolution (upsert/conflict modes)	Multi-agent divergence
R4	Tool security ($\text{tools}_{\text{eff}} = \text{tools}_c \cap \text{tools}_p$)	Privilege escalation
R5	Versioning (supersedes, soft-delete)	Provenance, audit

11.3 Key Results

From our axioms, mechanisms, and production measurements:

- **C1**: A1 + A3 + R3 ⇒ knowledge is durable and shared (3 scopes, 3–5% conflict rate, elimination traceable).
- **C2**: A2 + A5 + R2 ⇒ cost proportional to work. Production: boot drops from 73% to 24% of spawn cost.
- **C3**: A4 + R1 ⇒ 88% boot token reduction via `BuildSpawnContext()`, saving $2K–10K/month.
- **C4**: A6 + R3 ⇒ 6 agents sharing project memory achieve 3× faster knowledge coverage than solo agents.

11.4 Limitations

- Convergence (A4) is empirical, not proven — LLM noise can prevent it.
- Budget pruning may discard relevant low-priority knowledge under extreme pressure.
- Context memory write redundancy is ∼60% — most cycle-state writes repeat identical structure.

11.5 Future Work

- Formal convergence proof under bounded noise assumptions.
- Cross-relay federation for multi-organization agent networks.
- Memory patch mode (`set_memory` with diff/delta) to reduce write amplification on context memories.
- Auto-TTL for ephemeral context memories (expire after N cycles).

The agent does not live. It spawns, works, learns, dies. The next one knows everything.

References

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

LangChain, Inc. LangGraph: Build resilient language agents as graphs. `https://github.com/langchain-ai/langgraph`, 2024. Accessed: March 2026.

CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2024. Accessed: March 2026.

Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, 15(12):3535–3547, 2022. doi: 10.14778/3554821.3554842.

Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024. Open standard for LLM-tool integration. Accessed: March 2026.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. ACM, 2023.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2024.